

Perceptron: Building it from scratch in python



Becaye Baldé · [Follow](#)

5 min read · Jan 29, 2023



26



3

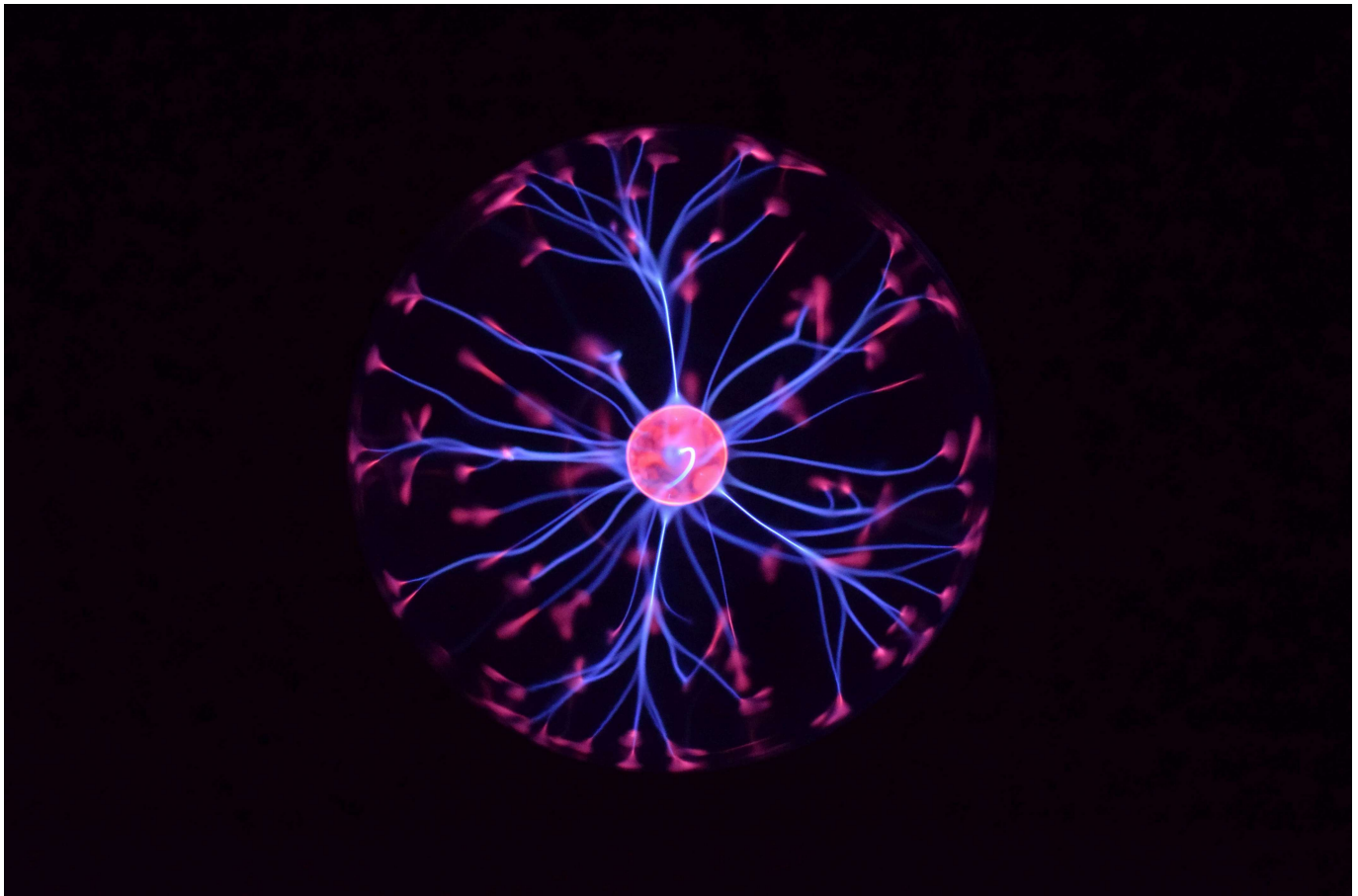


Photo by [Stefano Bucciarelli](#) on [Unsplash](#)

In this tutorial, we will build a custom Perceptron from scratch, then test it on the overused Iris dataset ;).

I assume that you have a theoretical understanding of the Perceptron. If not, please refer to my previous article: [Perceptron: The Cornerstone of Neural](#)

Open in app ↗

Sign up

Sign in

Medium

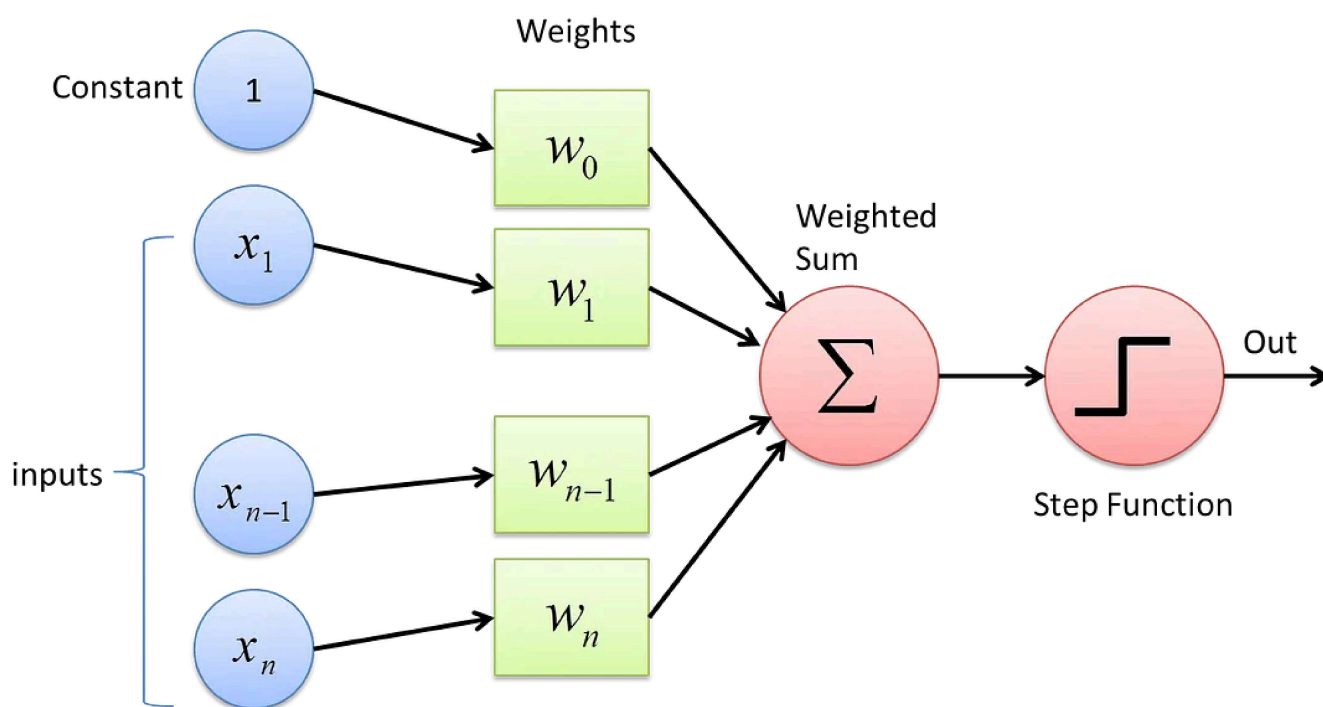
Search

Write



- Find the code on my Github: <https://github.com/BecayeSoft/Perceptron>.

Now let's build that Perceptron.



Anatomy of the Perceptron

We will follow the same steps as the figure above.

Step 1 — Perceptron Class

Follow these simple steps:

- Create a file called “Perceptron.py”
- Define a class called Perceptron inside
- Define a constructor inside the class

```
import numpy as np

class Perceptron(object):

    def __init__(self, eta=0.01, n_iter=10):
        self.eta = eta
        self.n_iter = n_iter
```

To build a perceptron, we need 3 attributes:

- η (eta): the learning rate is usually a small value between 0.0 and 1.0 which defines how quickly the model learns.
- **n_iter**: the number of iterations.
- **w_**: the weights. A weight defines the importance of the corresponding input value.

A learning rate too big may cause the model to converge too quickly without being optimal while a small learning rate may cause the model to get stuck.

Here is a good article about the learning rate.

Step 2 — Weighted sum

```
def weighted_sum(self, X):  
    return np.dot(X, self.w_[1:]) + self.w_[0]
```

`np.dot(X, w_)` is simply:

$$\sum_{i=0}^n X_i \cdot W_i$$

Step 3 — Step function

```
def predict(self, X):  
    return np.where(self.weighted_sum(X) >= 0.0, 1, -1)
```

The step function `predict()` returns:

- 1 — if the weighted sum is greater than 0.0
- -1 — Otherwise.

Step 4— Create the training method

An explanation will follow.

```

def fit(self, X, y):

    # initializing the weights to 0
    self.w_ = np.zeros(1 + X.shape[1])
    self.errors_ = []

    print("Weights:", self.w_)

    # training the model n_iter times
    for _ in range(self.n_iter):
        error = 0

        # loop through each input
        for xi, y in zip(X, y):

            # 1. calculate  $\hat{y}$  (the predicted value)
            y_pred = self.predict(xi)

            # 2. calculate Update
            # update =  $\eta * (y - \hat{y})$ 
            update = self.eta * (y - y_pred)

            # 3. Update the weights
            #  $W_i = W_i + \Delta(W_i)$  where  $\Delta(W_i) = \eta * (y - \hat{y}) = \text{update} * X_i$ 
            self.w_[1:] = self.w_[1:] + update * xi
            print("Updated Weights:", self.w_[1:])

            # Update the bias ( $X_0 = 1$ )
            self.w_[0] = self.w_[0] + update

            # if update != 0, it means that  $\hat{y} \neq y$ 
            error += int(update != 0.0)

        self.errors_.append(error)

    return self

```

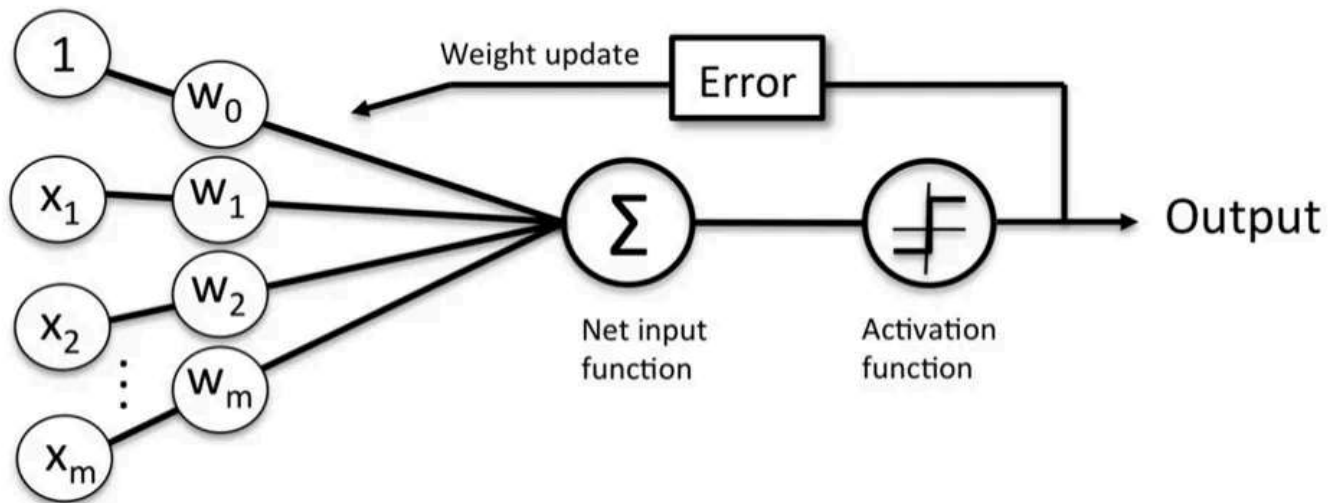
We initialize the weights to 0, then start our training.

For each input:

1. we calculate $\hat{y}$, the predicted output.

2. then we update the weights from W_1 to W_n (remember W_0 is the bias).

We repeat this process n_iter times (n_iter is the number of iterations).



Weights update in the perceptron

How are the weights updated?

Every weight W_i is updated by $\Delta(W_i)$.

$$W_i = W_i + \Delta(W_i)$$

Where:

$$\Delta(W_i) = \eta (y - \hat{y}) * X_i$$

Don't panic, it's easier than it seems:

- W_i is i -th weight.
- $\Delta(W_i)$ is the update. It determines if the weight increases or decreases.
- η is the learning rate.

- y is the actual value.
- $\hat{y}$ is the predicted value.
- X_i is the i -th input.

Example

Get your pen, we are going to do some math.

η	y	$\hat{y}$	X_i
0.01	1	1	0.5
0.01	-1	-1	1
0.01	1	-1	4
0.01	-1	1	2

Random data to calculate weights update

$$\Delta(W_i) = \eta (y - \hat{y}) * X_i$$

When y is equal to $\hat{y}$, there is no update:

- $\Delta(W_1) = 0.01 * (1 - 1) * 0.5 = 0$
- $\Delta(W_2) = 0.01 * ((-1) - (-1)) * 1 = 0$

When y is different than $\hat{y}$, it means that there's an error and the weights must be updated by $\Delta(W_i)$:

- $\Delta(W3) = 0.01 * (1 - (-1)) * 4 = 0.08$
- $\Delta(W4) = 0.01 * ((-1) - 1) * 2 = -0.04$

Whew!

We made it through to the end, let's test our Perceptron.

Step 5 — Testing the Perceptron

The iris dataset contains 3 classes of 50 instances each, where each class refers to a type of iris plant.

```
import pandas as pd
from sklearn.utils import shuffle
import numpy as np

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/iris')
# Shuffling the data
df = shuffle(df)

df.head()
```

	0	1	2	3	4	
65	6.7	3.1	4.4	1.4		Iris-versicolor
86	6.7	3.1	4.7	1.5		Iris-versicolor
89	5.5	2.5	4.0	1.3		Iris-versicolor
147	6.5	3.0	5.2	2.0		Iris-virginica
1	4.9	3.0	1.4	0.2		Iris-setosa
13	4.3	3.0	1.1	0.1		Iris-setosa

Separating the data (X) from the labels (y)


```
X = df.iloc[:, 0:4].values
y = df.iloc[:, 4].values

print(X[0:5])
print(y[0:5])
```

```
[[4.6 3.2 1.4 0.2]
 [5.8 2.7 4.1 1. ]
 [5.7 2.5 5.  2. ]
 [5.6 2.5 3.9 1.1]
 [6.5 3.  5.8 2.2]]

['Iris-setosa' 'Iris-versicolor' 'Iris-virginica' 'Iris-versicolor' 'Iris-virginica']
```

Splitting the data into train and test set

```
# 75% for train and 25% for test
train_data, test_data, train_labels, test_labels = train_test_split(
    X, y, test_size=0.25)

# Encoding the labels: 1 for setosa, -1 otherwise
train_labels = np.where(train_labels == 'Iris-setosa', 1, -1)
test_labels = np.where(test_labels == 'Iris-setosa', 1, -1)

print('Train data:', train_data[0:2])
print('Train labels:', train_labels[0:5])

print('Test data:', test_data[0:2])
print('Test labels:', test_labels[0:5])
```

```
Train data: [[4.6 3.2 1.4 0.2]
 [5.5 2.6 4.4 1.2]]
```

```
Train labels: [1 0 1 0 0]
```

```
Test data: [[7.2 3.6 6.1 2.5]
            [5.7 3.8 1.7 0.3]]

Test labels: [0 1 0 0 1]
```

Training the perceptron

```
perceptron = p.Perceptron(eta=0.1, n_iter=10)
perceptron.fit(train_data, train_labels)
```

Making predictions on test data

```
test_preds = perceptron.predict(test_data)
print(test_preds)
```

```
[0 1 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 0 1 0 0 0 1 1 0 1 0 0 0 0 0 1 0 1
 0]
```

Measuring Performances

```
accuracy = accuracy_score(y_preds, test_labels)
print('Accuracy:', round(accuracy, 2) * 100, "%")
```

Accuracy: 100.0 %

Bonus: Perceptron with Sklearn

The purpose of implementing a Perceptron from scratch was to grasp the underlying concepts. But you could have done that easily using Sickit-learn.

```
from sklearn.linear_model import Perceptron

perceptron = Perceptron(random_state = 42, max_iter = 20, tol = 0.001)
perceptron.fit(train_data, train_labels)
test_preds = setosa_perceptron.predict(test_data)
```

```
test_accuracy = accuracy_score(test_preds, test_labels)
print("Accuracy on test data: ", round(test_accuracy, 2) * 100, "%")
```

Accuracy on test data: 100.0 %

You can find the code on my [GitHub](#).

We have reached the end of the tutorial, please report any problem in the comments.

Stay tuned for more posts ;).